demo.py

```python

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ----------------------------
# 1. Project configuration
# ----------------------------
BASE_DIR = Path(__file__).resolve().parent
CSV_PATH = BASE_DIR / "sales_data.csv"
sns.set_theme(style="whitegrid")


# ----------------------------
# 2. Generate sample data
# ----------------------------
def generate_sales_data():
    rng = np.random.default_rng(42)
    rows = 500

    products = {
        "Laptop": 55000,
        "Mobile": 25000,
        "Headphones": 2000,
        "Keyboard": 1500,
        "Monitor": 12000,
    }

    product_names = list(products.keys())
    chosen_products = rng.choice(product_names, rows)
    quantities = rng.integers(1, 6, rows)

    df = pd.DataFrame({
        "Order_ID": range(1001, 1001 + rows),
        "Date": rng.choice(
            pd.date_range("2026-01-01", "2026-10-28", freq="D"),
            rows
        ),
        "Product": chosen_products,
        "Quantity": quantities,
        "Unit_Price": [
            products[product] for product in chosen_products
        ],
        "Region": rng.choice(
            ["Delhi", "Mumbai", "Bangalore", "Chennai"],
            rows
        ),
    })
```

```
52
53      df["Revenue"] = df["Quantity"] * df["Unit_Price"]
54      df.to_csv(CSV_PATH, index=False)
55      print(f"Sample dataset created: {CSV_PATH.name}")
56      return df
57
58
59  # ----------------------------
60  # 3. Load and clean data
61  # ----------------------------
62  def load_data():
63      if CSV_PATH.exists():
64          df = pd.read_csv(CSV_PATH)
65      else:
66          df = generate_sales_data()
67
68      df.columns = df.columns.str.strip()
69      df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
70
71      for column in ["Quantity", "Unit_Price"]:
72          df[column] = pd.to_numeric(df[column], errors="coerce")
73
74      df = df.dropna(
75          subset=["Date", "Product", "Quantity", "Unit_Price", "Region"]
76      )
77      df = df[(df["Quantity"] > 0) & (df["Unit_Price"] >= 0)]
78
79      # Recalculate revenue to keep the calculation consistent.
80      df["Revenue"] = df["Quantity"] * df["Unit_Price"]
81      df["Month"] = df["Date"].dt.to_period("M").astype(str)
82
83      if df.empty:
84          raise ValueError("No valid sales records found in the CSV file.")
85
86      return df
87
88
89  # ----------------------------
90  # 4. Analyze sales
91  # ----------------------------
92  def analyze_sales(df):
93      total_revenue = df["Revenue"].sum()
94      total_orders = df["Order_ID"].nunique() if "Order_ID" in df else len(df)
95      total_units = df["Quantity"].sum()
96      average_order_value = total_revenue / total_orders if total_orders else 0
97
98      print("\n" + "=" * 45)
99      print("       SALES DATA ANALYTICS REPORT")
100     print("=" * 45)
101     print(f"Total Revenue       : Rs. {total_revenue:,.2f}")
102     print(f"Total Orders        : {total_orders:,}")
103     print(f"Units Sold          : {total_units:,.0f}")
104     print(f"Average Order Value : Rs. {average_order_value:,.2f}")
105
```

```
    product_sales = (
        df.groupby("Product", as_index=False)["Revenue"]
        .sum()
        .sort_values("Revenue", ascending=False)
    )

    region_sales = (
        df.groupby("Region", as_index=False)["Revenue"]
        .sum()
        .sort_values("Revenue", ascending=False)
    )

    monthly_sales = (
        df.groupby("Month", as_index=False)["Revenue"]
        .sum()
        .sort_values("Month")
    )

    print("\nProduct-wise Revenue:")
    print(product_sales.to_string(index=False))

    print("\nRegion-wise Revenue:")
    print(region_sales.to_string(index=False))

    print("\nMonthly Revenue:")
    print(monthly_sales.to_string(index=False))

    product_sales.to_csv(BASE_DIR / "product_sales_report.csv", index=False)
    region_sales.to_csv(BASE_DIR / "region_sales_report.csv", index=False)
    monthly_sales.to_csv(BASE_DIR / "monthly_sales_report.csv", index=False)

    return product_sales, region_sales, monthly_sales


# ----------------------------
# 5. Create dashboard charts
# ----------------------------
def create_dashboard(df, product_sales, region_sales, monthly_sales):
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    sns.barplot(
        data=product_sales,
        x="Product",
        y="Revenue",
        ax=axes[0, 0]
    )
    axes[0, 0].set_title("Revenue by Product")
    axes[0, 0].tick_params(axis="x", rotation=20)
    axes[0, 0].set_ylabel("Revenue (Rs.)")

    sns.barplot(
        data=region_sales,
        x="Region",
        y="Revenue",
```

```
        ax=axes[0, 1]
    )
    axes[0, 1].set_title("Revenue by Region")
    axes[0, 1].set_ylabel("Revenue (Rs.)")

    sns.lineplot(
        data=monthly_sales,
        x="Month",
        y="Revenue",
        marker="o",
        ax=axes[1, 0]
    )
    axes[1, 0].set_title("Monthly Revenue Trend")
    axes[1, 0].tick_params(axis="x", rotation=45)
    axes[1, 0].set_ylabel("Revenue (Rs.)")

    sns.countplot(
        data=df,
        x="Product",
        order=df["Product"].value_counts().index,
        ax=axes[1, 1]
    )
    axes[1, 1].set_title("Number of Sales Records by Product")
    axes[1, 1].tick_params(axis="x", rotation=20)
    axes[1, 1].set_ylabel("Records")

    fig.suptitle("Sales Data Analytics Dashboard", fontsize=18)
    fig.tight_layout()

    output_path = BASE_DIR / "sales_dashboard.png"
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    print(f"\nDashboard saved: {output_path.name}")

    plt.show()


# ----------------------------
# 6. Run the project
# ----------------------------
if __name__ == "__main__":
    sales_df = load_data()
    print(f"\nValid records analyzed: {len(sales_df)}")

    product_report, region_report, monthly_report = analyze_sales(sales_df)
    create_dashboard(
        sales_df, product_report, region_report, monthly_report
    )

    print("\nProject completed successfully!")


```